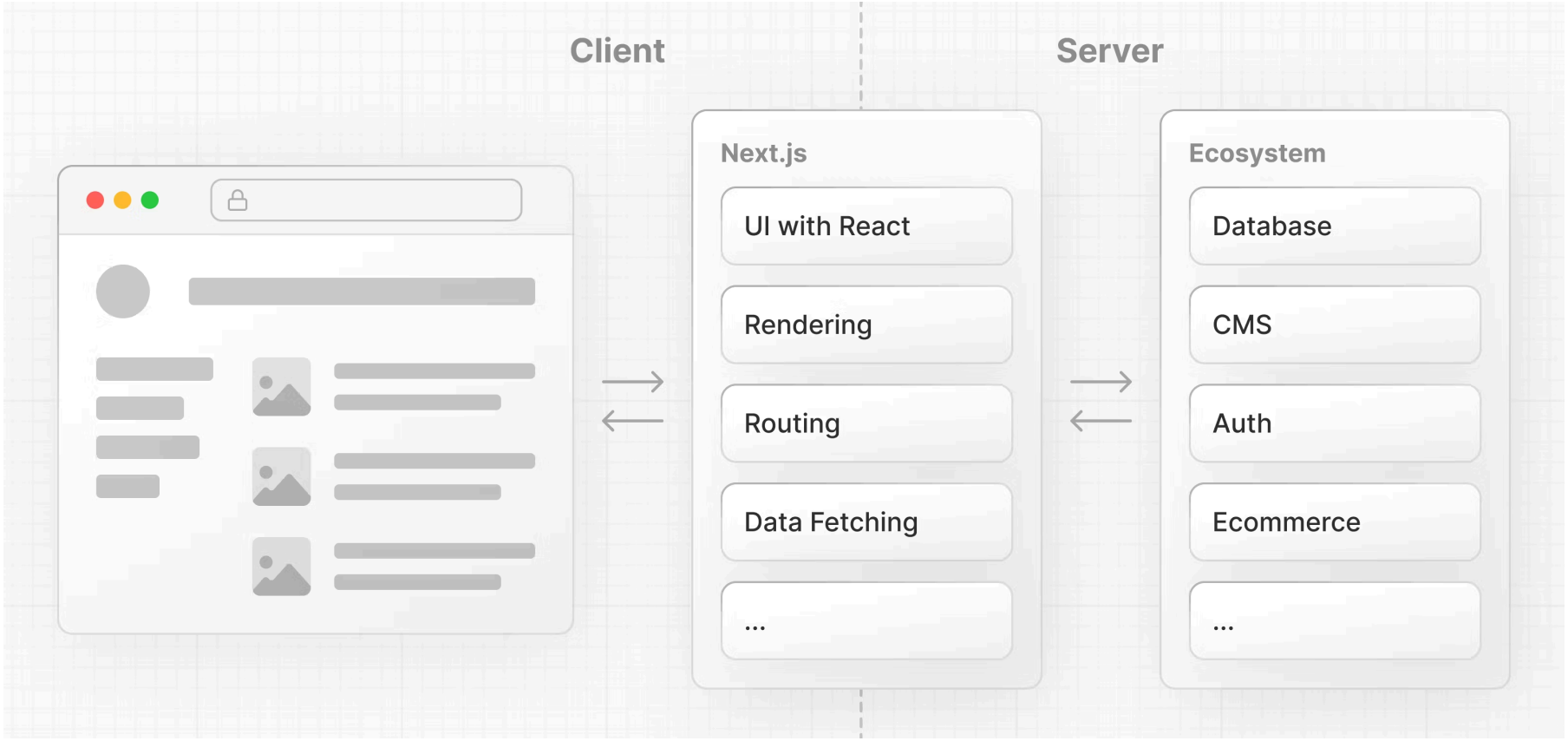
Next.js Intro



→ What is Next.js ?

- Next.js is a React framework that gives you building blocks to create web applications.



→ Installing Next.js

- Automatic Installation

```
npx create-next-app@latest
```

- Manual Installation

```
npm install react@latest react-dom@latest next@latest
```

- System Requirements:
  - Node.js 20.9 or higher
  - macOS, Windows (including WSL), and Linux are supported.

> ℹ️ Installation

→ Project Structure 📁

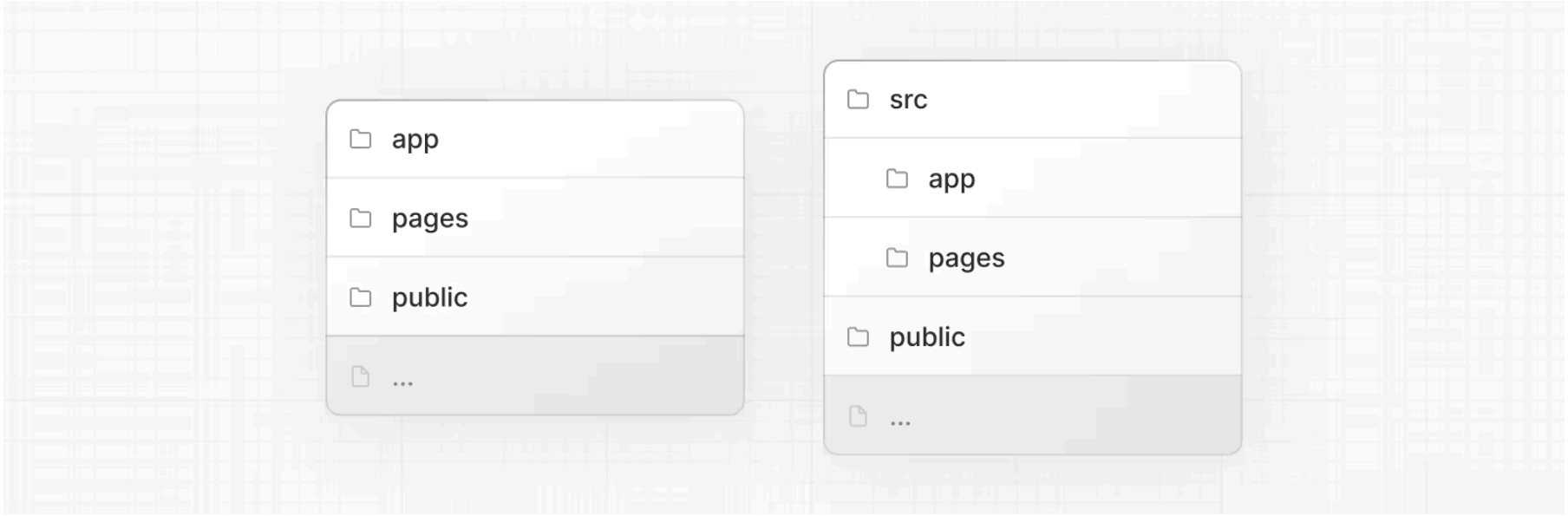
package.json

```
"scripts": {
   "dev": "next dev",
   "build": "next build",
   "start": "next start",
   "lint": "next lint"
 },
```

- dev: runs next dev to start Next.js in development mode.
- build: runs next build to build the application for production usage.
- start: runs next start to start a Next.js production server.
- lint: runs next lint to set up Next.js' built-in ESLint configuration.

→ Project Structure

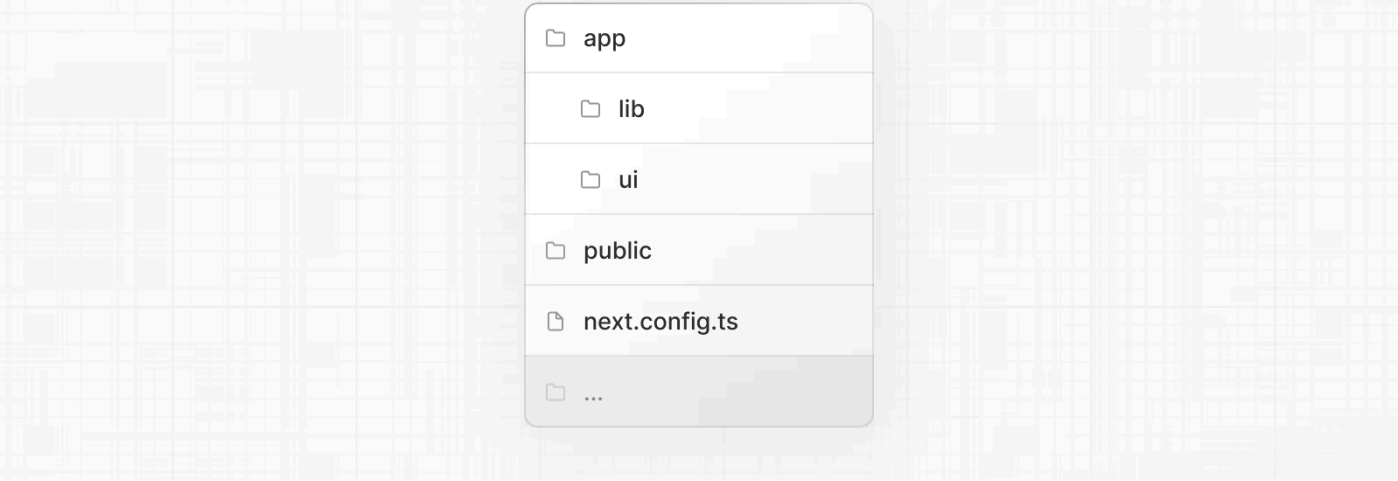
Top-level folders



- `app` App Router
- `pages` Pages Router
- `public` Static assets to be served
- `src` Optional application source folder

> ℹ Next.js uses file-system routing, which means the routes in your application are determined by how you structure your files.

→ Next.js Project Structure 📁



- `app` → App Router, components, logic
  - `lib` → functions e.g. Data fetching
  - `ui` → UI components
- `public` → Static assets to be served
- `config files` → next.config.ts

> ℹ️ Next.js uses file-system routing, which means the routes in your application are determined by how you structure your files.

→ app Routing Conventions

layout	.js .jsx .tsx	Layout
page	.js .jsx .tsx	Page
loading	.js .jsx .tsx	Loading UI
not-found	.js .jsx .tsx	Not found UI
error	.js .jsx .tsx	Error UI
global-error	.js .jsx .tsx	Global error UI
route	.js .ts	API endpoint
template	.js .jsx .tsx	Re-rendered layout
default	.js .jsx .tsx	Parallel route fallback page

→ app Routing Conventions

Nested Routes

- **folder** → Route segment
- **folder/folder** → Nested route segment

Dynamic Routes

- **[folder]** → Dynamic route segment
- **[...folder]** → Catch-all route segment
- **[[...folder]]** → Optional catch-all route segment

→ Create based on template

-  starter-example

```
npx create-next-app@latest nextjs-dashboard
--use-npm
--example
"https://github.com/vercel/next-learn/tree/main/dashboard/starter-example"
```

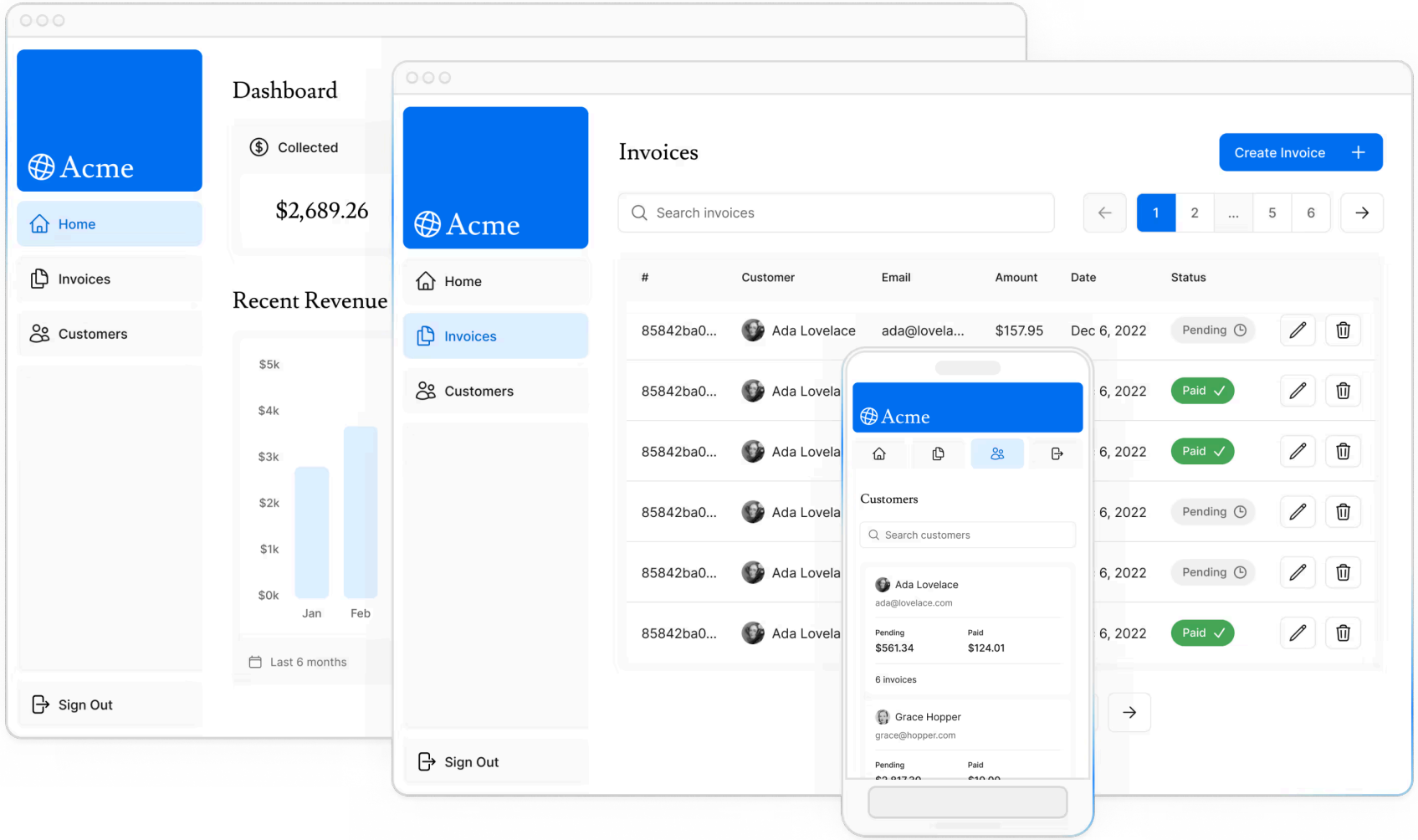
→ Placeholder data

- If a database or API is not yet available
  → placeholder data to build
  - JSON format
  - JavaScript objects
  - 3rd party service like mockAPI🔗

- `/app/lib/placeholder-data.ts` 📁

```
const invoices = [
  {
    customer_id: customers[0].id,
    amount: 15795,
    status: 'pending',
    date: '2022-12-06',
  },
  {
    customer_id: customers[1].id,
    amount: 20348,
    status: 'pending',
    date: '2022-11-14',
  },
  // ...
];
```

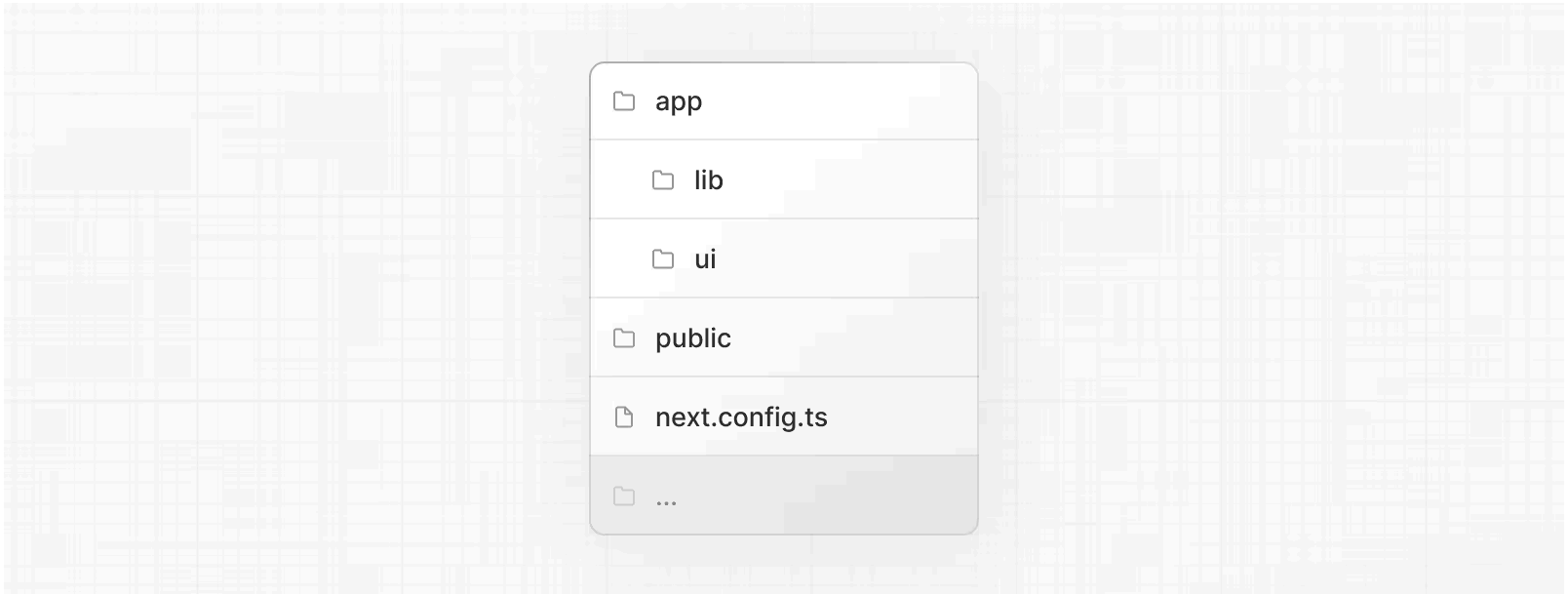
→ Example Project → dashboard-app



📑 Online Tutorial https://nextjs.org/learn/dashboard-app 🔗

→ Example Project

Structure



- `/app` → routes, components and logic for your application
- `/app/lib` → functions (such as reusable utility functions and data fetching functions)
- `/app/ui` → UI components for your application, (cards, tables, and forms)
- `/public` → static assets for your application (images)
- `/scripts` → seeding script (populate database)

→ Example Project

Run

- Running the development server 🚀

```
npm run dev
```

- http://localhost:3000/ 🔗

→ Example Project

Topics

- `Styling` → The different ways to style your application in Next.js.
- `Optimizations` → How to optimize images, links, and fonts.
- `Routing` → How to create nested layouts and pages using file-system routing.
- `Data Fetching` → How to set up a database on Vercel, and best practices for fetching and streaming.
- `Search and Pagination` → How to implement search and pagination using URL Search Params.
- `Mutating Data` → How to mutate data using React Server Actions, and revalidate the Next.js cache.

→ Example Project → Topics → CSS Styling 🕴️

- `/app/layout.tsx` 📁

```tsx
import '@/app/ui/global.css';

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

- `/app/ui/global.css` 📁

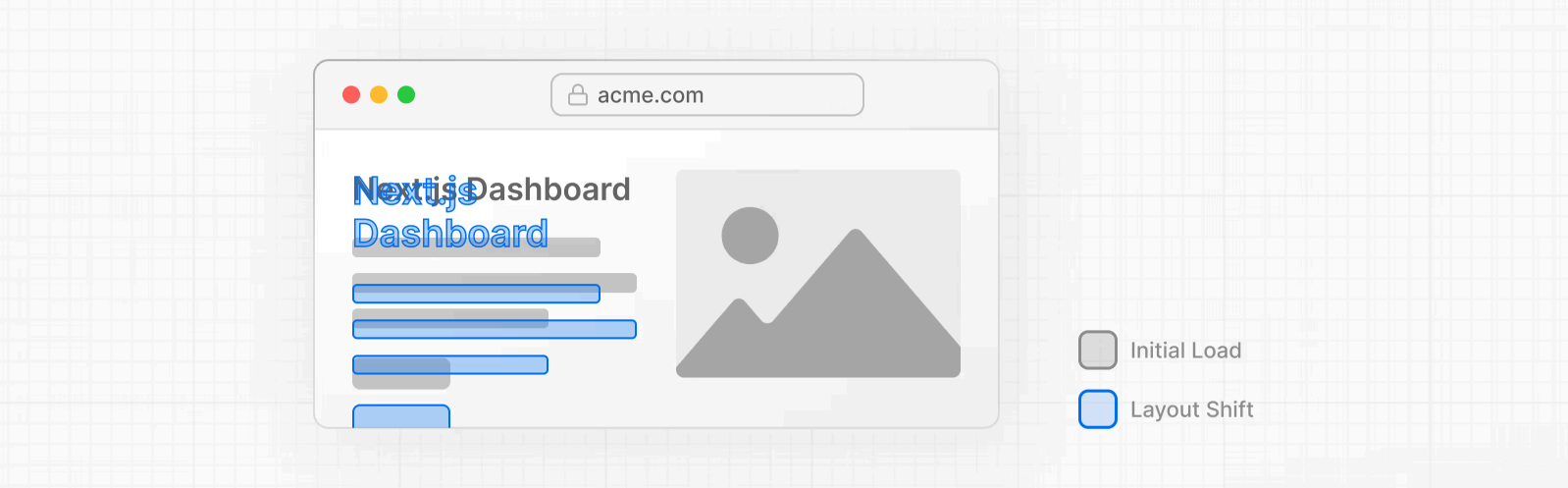
```css
@tailwind base;
@tailwind components;
@tailwind utilities;
```

> ℹ️ Tailwind 🔗 is a CSS framework

```tsx
<h1 className="text-blue-500">I'm blue!</h1>
```

> 😕 look at the `/app/page.tsx` 📁

layout shift



- when the browser initially renders text in a fallback or system font and then swaps it out for a custom font once it has loaded.
  - this **swap** can cause the text size, spacing, or layout to change, shifting elements around it.
- → automatically optimizes fonts with `next/font` module

Adding fonts

- `/app/ui/fonts.ts`

```
import { Inter, Lusitana } from 'next/font/google';

export const inter = Inter({ subsets: ['latin'] });

export const lusitana = Lusitana({
  weight: ['400', '700'],
  subsets: ['latin'],
});
```

- `/app/layout.tsx`

```
import '@/app/ui/global.css';
import { inter } from '@/app/ui/fonts';
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="en">
      <body
      className={`${inter.className}
      antialiased`}>{children}</body>
    </html>
  );
}
```

Adding fonts

- `/app/page.tsx`

```
...
import Link from 'next/link';
import { lusitana } from '@/app/ui/fonts';

export default function Page() {
  return (
    // ...
    <p
      className={`${lusitana.className} text-xl text-gray-800 md:text-3xl md:leading-normal`}
    >
      <strong>Welcome to Acme.</strong> This is the example for the{' '}
      <a href="https://nextjs.org/learn/" className="text-blue-500">
        Next.js Learn Course
      </a>
    // ...
  );
}
```

Optimize images

- Regular HTML

```
<img
  src="/hero.png"
  alt="Screenshots of the dashboard project showing desktop version"
/>
```

👨‍🔧 `Manually`

- Ensure your image is responsive on different screen sizes.
- Specify image sizes for different devices.
- Prevent layout shift as the images load.
- Lazy load images that are outside the user's viewport.

- The `Image` component 📦
  - Preventing layout shift automatically when images are loading.
  - Resizing images to avoid shipping large images to devices with a smaller viewport.
  - Lazy loading images by default (images load as they enter the viewport).
  - Serving images in modern formats, like **WebP** and **AVIF**, when the browser supports it.

Optimize images

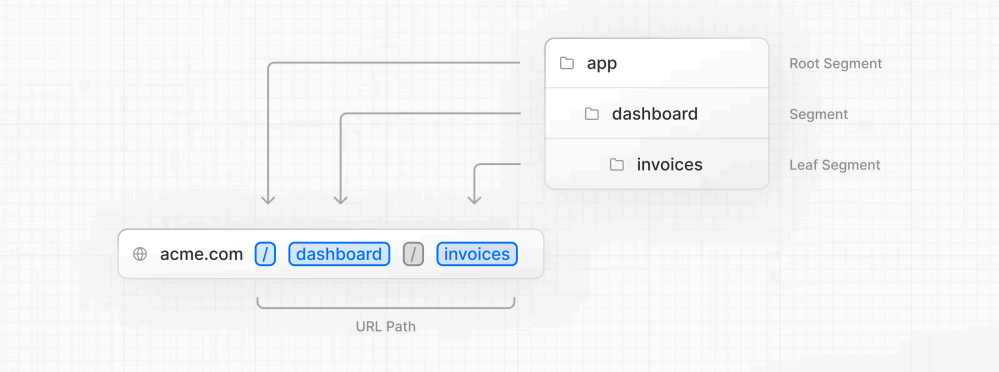
- `/app/page.tsx`

```
...
import { lusitana } from '@/app/ui/fonts';
import Image from 'next/image';

export default function Page() {
  return (
    // ...
    <div className="flex items-center justify-center p-6 md:w-3/5 md:px-28 md:py-12">
      {/* Add Hero Images Here */}
      <Image
        src="/hero-desktop.png" width={1000}  height={760}  className="hidden md:block"
        alt="Screenshots of the dashboard project showing desktop version"
      />        //...
  );
}
```

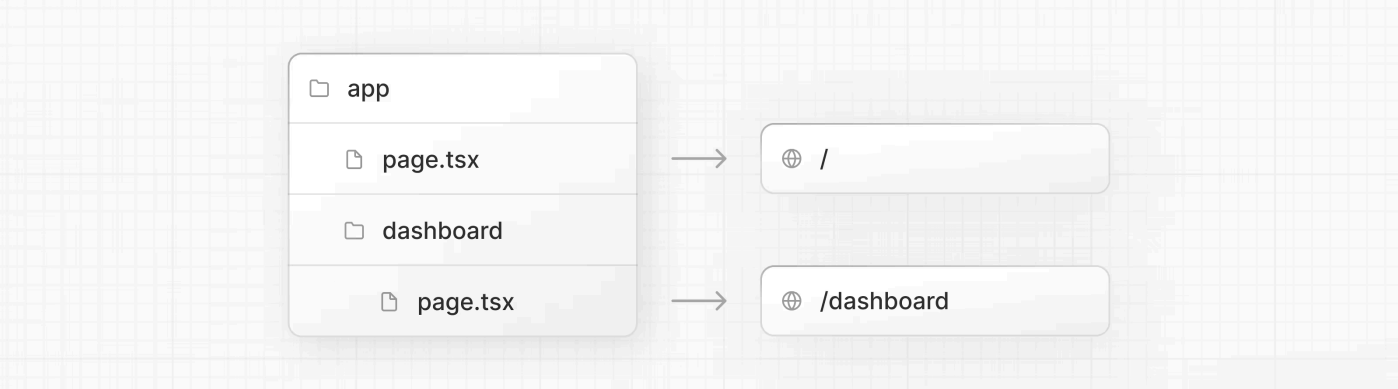
> 🤦 the class **hidden** to remove the image from the DOM on mobile screens, and **md:block** to show the image on desktop screens.

Nested routing



- uses file-system routing
  - folders are used to create nested routes
  - each folder represents a route segment that maps to a URL segment
- You can create separate UIs for each route using **layout.tsx** and **page.tsx** files.
- **page.tsx** → special next js file that exports a React component - **required for the route to be accessible**
- `app/page.tsx` → the home page associated with the route **/**

Nested routing

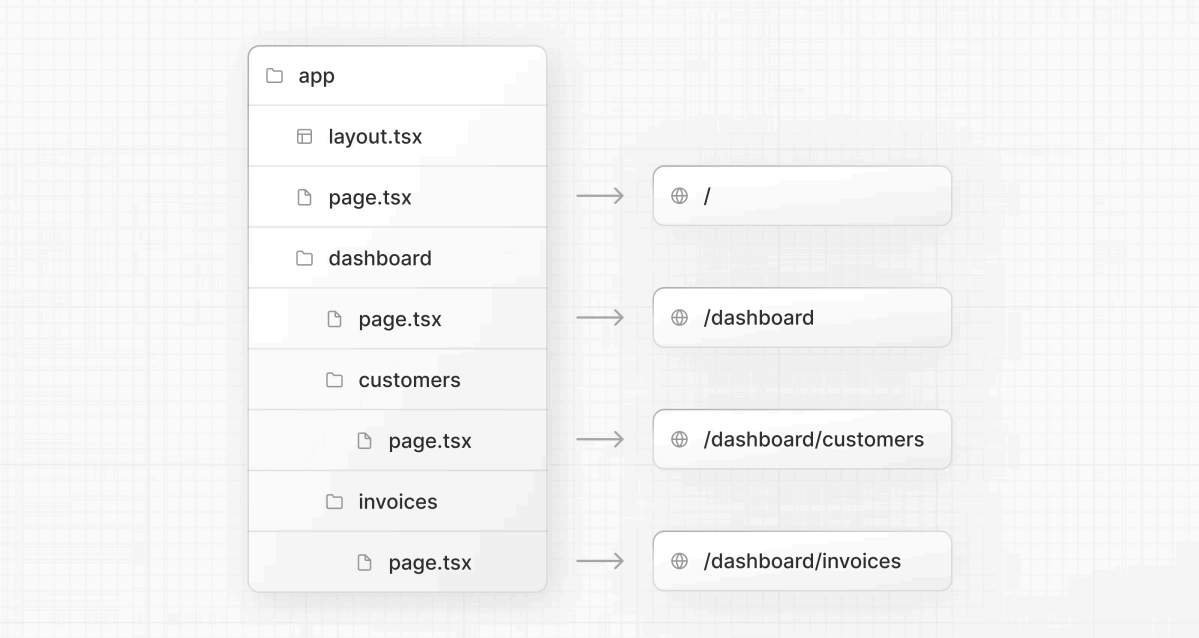


- `/app/dashboard/page.tsx`

```
export default function Page() {
  return <p>Dashboard Page</p>;
}
```

- http://localhost:3000/dashboard

Nested routing



- `/app/dashboard/customers/page.tsx`

```
export default function Page() {
  return <p>Customers Page</p>;
}
```

- `/app/dashboard/invoices/page.tsx`

```
export default function Page() {
    return <p>Invoices Page</p>;
  }
```

Creating the dashboard layout

> `layout.tsx` → file to create UI that is shared between multiple pages

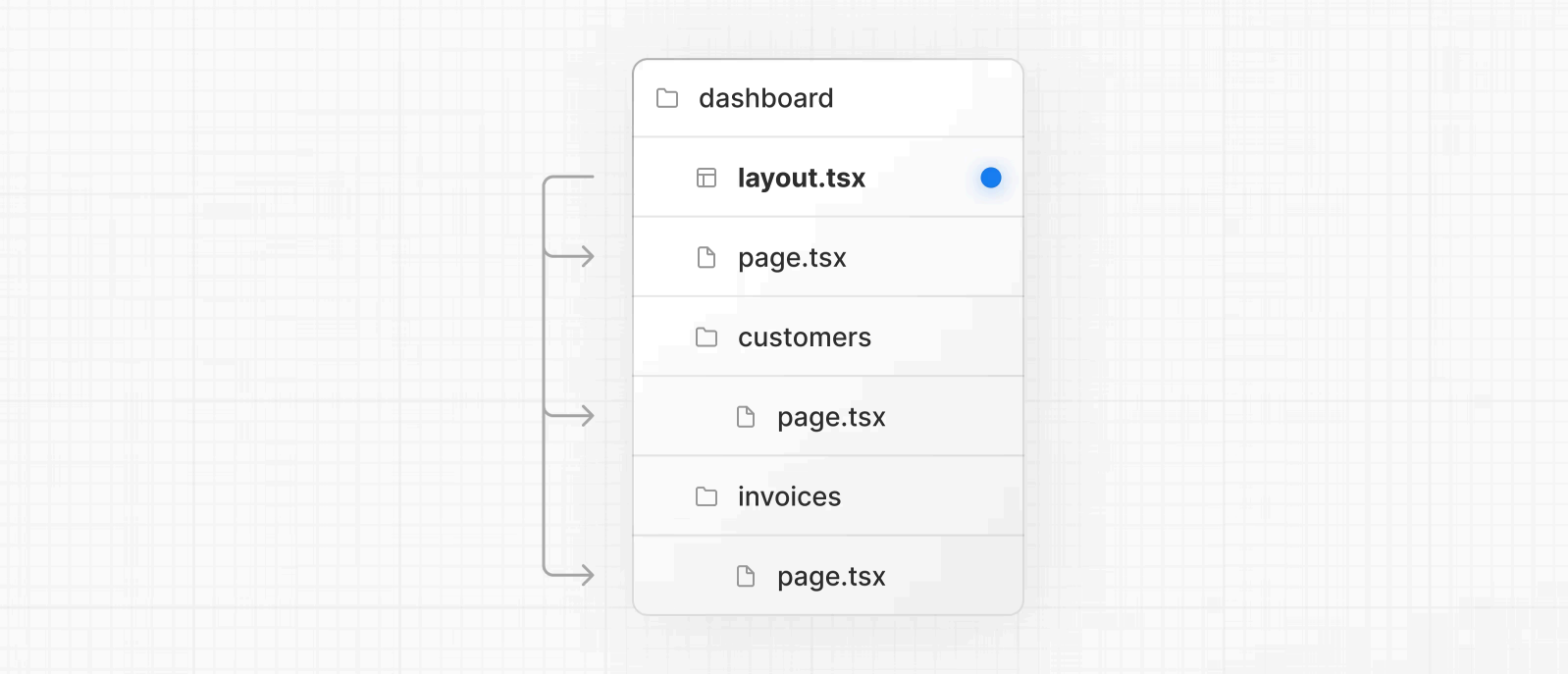
- `/app/dashboard/layout.tsx`

```tsx
import SideNav from '@/app/ui/dashboard/sidenav';

export default function Layout({ children }: { children: React.ReactNode }) {
  return (
    <div className="flex h-screen flex-col md:flex-row md:overflow-hidden">
      <div className="w-full flex-none md:w-64">
        <SideNav />
      </div>
      <div className="flex-grow p-6 md:overflow-y-auto md:p-12">{children}</div>
    </div>
  );
}
```

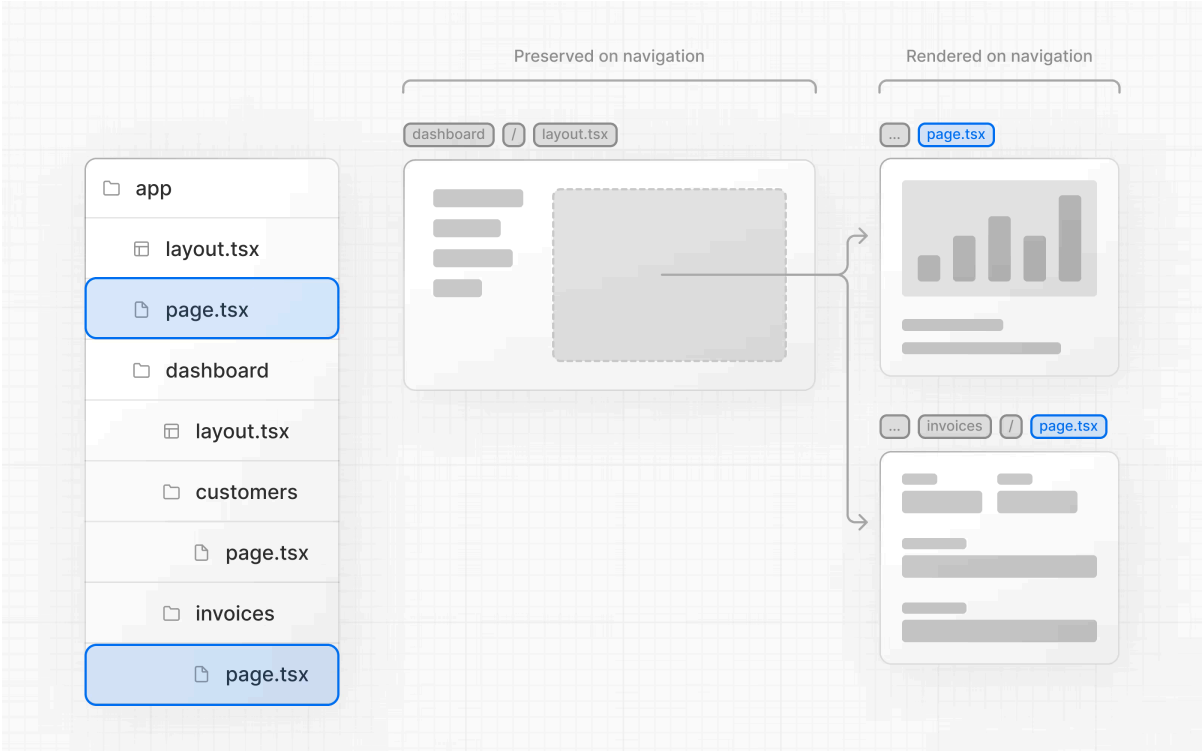
Creating the dashboard layout

- importing the **<SideNav />** 📦 component into your layout.
- The **<Layout />** 📦 component receives a children prop (page or another layout)
  - pages inside /dashboard will automatically be nested inside a **<Layout />**



partial rendering

- only the route segments that change on navigation re-render on the client, and any shared segments are preserved.



Root Layout

- /app/layout.tsx
  - root layout will be shared across all pages in your application
    - use the root layout to modify `<html>` and `<body>` tags, and add `metadata`

Optimize Navigation

- `<a>` HTML element
  - `<a href="…"></a>`
    - full page refresh on each page navigation!

- The "**Link**" 📦 component
  - link between pages in your application
  - client-side navigation with JavaScript.
  - `<Link key={link.name} href={link.href}>`

→ Example Project → Topics → Navigating

- `/app/ui/dashboard/nav-links.tsx`

from

```
<a
            key={link.name}
            href={link.href}
            className="flex h-[48px] grow items-center justify-center gap-2 rounded-md bg-gray-50 p-3 text-sm font-medium hover:bg-sky-100 hover:text-blue-600 md:flex-none md:justify-start md:p-2 md:px-3"
          >
```

to

```
import Link from 'next/link';

          <Link
            key={link.name}
            href={link.href}
            className="flex h-[48px] grow items-center justify-center gap-2 rounded-md bg-gray-50 p-3 text-sm font-medium hover:bg-sky-100 hover:text-blue-600 md:flex-none md:justify-start md:p-2 md:px-3"
          >
            <LinkIcon className="w-6" />
            <p className="hidden md:block">{link.name}</p>
          </Link>
```

- traditional React **SPA** → the browser loads all your application code on initial load
- whenever components appear in the browser's viewport
  - Next.js automatically **prefetches** the code for the linked route in the background

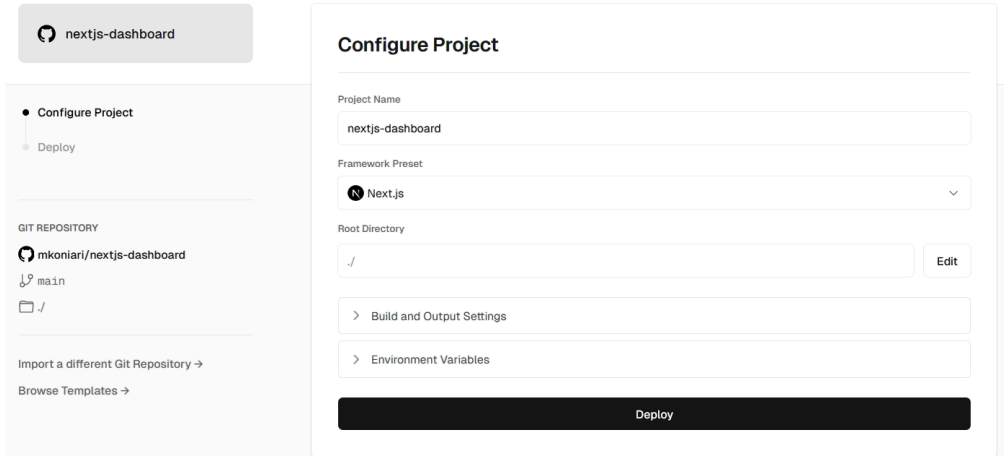
Showing active links

- user's current path from the URL → hook **usePathname()**
  - `nav-links.tsx` 📁 → **Client Component**
    - 'use client';

```
'use client';
import {
  UserGroupIcon,
  HomeIcon,
  InboxIcon,
} from '@heroicons/react/24/outline';
import Link from 'next/link';
import { usePathname } from 'next/navigation';
import clsx from 'clsx';
...

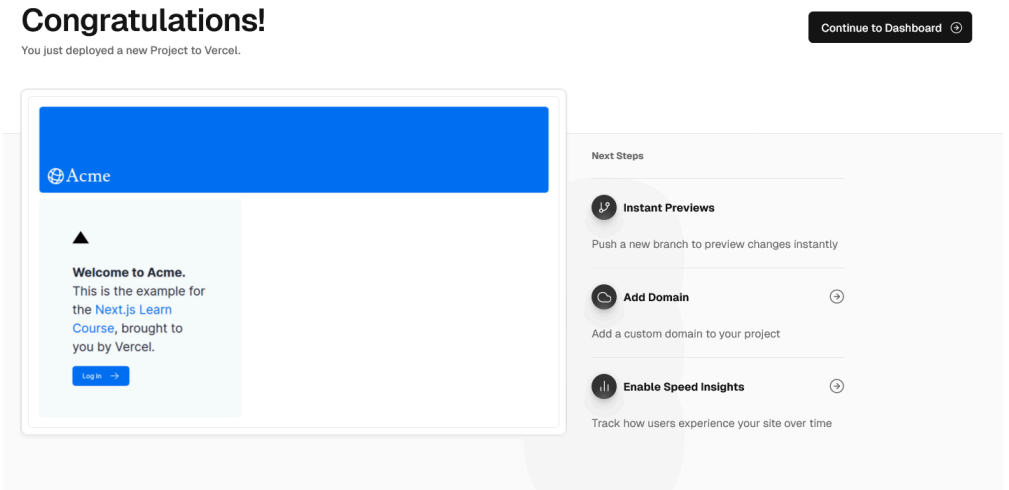
export default function NavLinks() {
  const pathname = usePathname();
  return (
...
 className={clsx(
              'flex h-[48px] grow items-center justify-center gap-2 rounded-md bg-gray-50 p-3 text-sm font-medium hover:bg-sky-100 hover:text-blue-600 md:flex-none md:justify-start md:p-2 md:px-3',
              {
                'bg-sky-100 text-blue-600': pathname === link.href,
              },
            )}
```

Setting Up a Database

- Set up Github🔗
- Set up a Vercel account🔗
- Connect and deploy your project (link your GitHub repo)
  - 

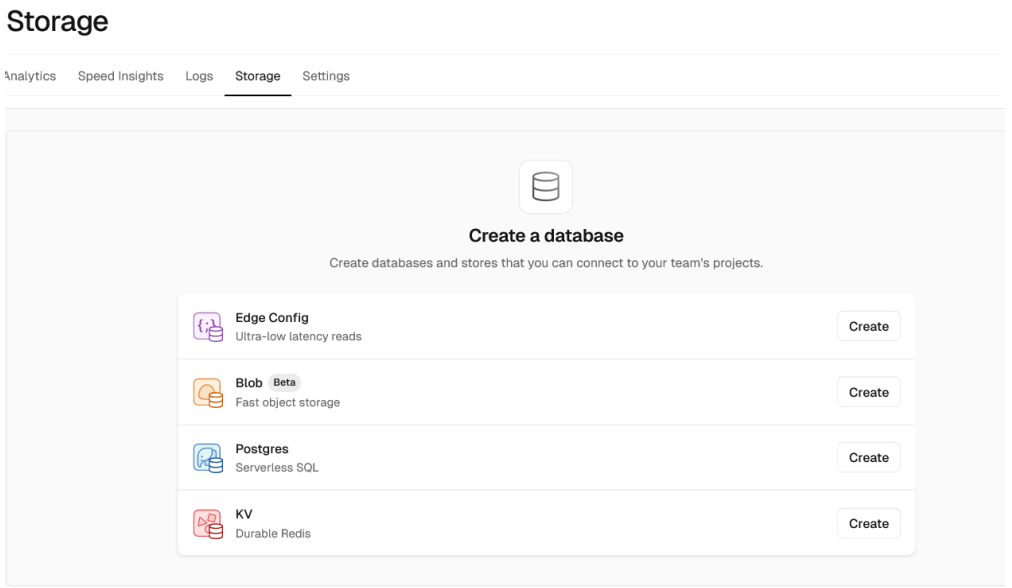
> visit live site - whenever you push changes to your main branch, Vercel will automatically redeploy your application

Setting Up a Database

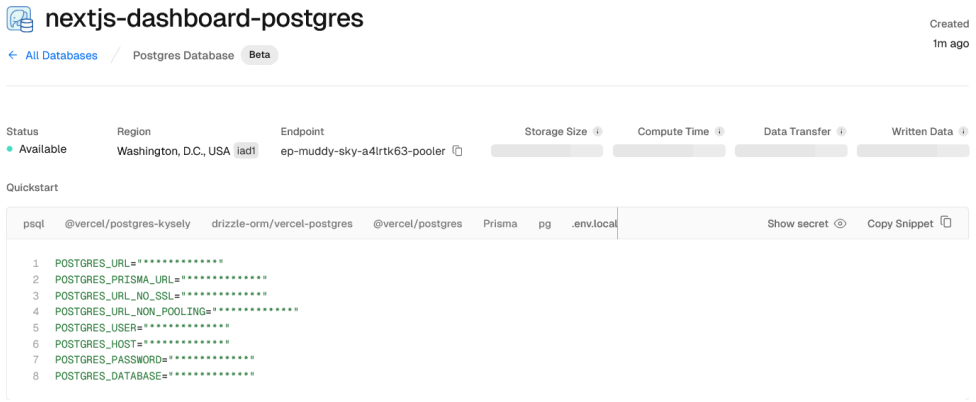
- Connect and deploy your project (link your GitHub repo)
  - 

Setting Up a Database

- Create and link your project to a database
  - Dashboard → Storage tab → Connect Store → Create New → Select a database provider → Continue.
    -



Setting Up a Database

- Create and link your project to a database
  - navigate to the .env.local tab, click Show secret and Copy Snippet
    - 

Setting Up a Database

- rename the **.env.example** file to **.env**
  - Paste in the copied contents from Vercel
  - **add to .ignore file (GitHub)** 🗡️
- install Vercel Postgres SDK

```
npm i @vercel/postgres
```

Setting Up a Database

- Seed the database with initial data.
- visit http://localhost:3000/seed or live hosted site/seed

> If Error 🛑 → view https://github.com/vercel/next-learn/issues/1022

Executing queries

- `app/query/route.ts` 📂
  - listInvoices()

```
SELECT invoices.amount, customers.name
FROM invoices
JOIN customers ON invoices.customer_id = customers.id
WHERE invoices.amount = 666;
```

1. Uncomment the file
2. Remove the Response.json() block
3. Navigate to http://localhost:3000/query 🔗

→ Example Project → Topics → Data Fetching

- React Server Components
  - support promises → use async/await syntax
  - execute on the server → send the result to the client
- `/app/lib/data.ts`

```
import postgres from 'postgres';

const sql = postgres(process.env.POSTGRES_URL!, { ssl: 'require' });
```

> Keep data queries in the data.ts file, and you can import them into the components.

Fetching data for the dashboard overview page

- `/app/dashboard/page.tsx`

```
import { Card } from '@/app/ui/dashboard/cards';
import RevenueChart from '@/app/ui/dashboard/revenue-chart';
import LatestInvoices from '@/app/ui/dashboard/latest-invoices';
import { lusitana } from '@/app/ui/fonts';

export default async function Page() {
  return (
    <main>
      <h1 className={`${lusitana.className} mb-4 text-xl md:text-2xl`}>Dashboard</h1>
      <div className="grid gap-6 sm:grid-cols-2 lg:grid-cols-4">
        {/* <Card title="Collected" value={totalPaidInvoices} type="collected" /> */}
        {/* <Card title="Pending" value={totalPendingInvoices} type="pending" /> */}
        {/* <Card title="Total Invoices" value={numberOfInvoices} type="invoices" /> */}
        {/* <Card title="Total Customers" value={numberOfCustomers} type="customers"/> */}</div>
      <div className="mt-6 grid grid-cols-1 gap-6 md:grid-cols-4 lg:grid-cols-8">
        {/* <RevenueChart revenue={revenue}  /> */}{/* <LatestInvoices latestInvoices={latestInvoices} /> */}
      </div></main> );}
```

Fetching data for the dashboard overview page

- `/app/dashboard/page.tsx`

```
...
import { lusitana } from '@/app/ui/fonts';
import { fetchRevenue } from '@/app/lib/data';

export default async function Page() {
    const revenue = await fetchRevenue();
  return (
    ...
         { <RevenueChart revenue={revenue}  /> }
    ...
  );
}
```

- `/app/ui/dashboard/revenue-chart.tsx` → uncomment all code
- visit http://localhost:3000/dashboard or vercel site

Fetching data for <LatestInvoices/>

- `/app/lib/data.ts`

```
// Fetch the last 5 invoices, sorted by date
const data = await sql<LatestInvoiceRaw>`
  SELECT invoices.amount, customers.name,
  customers.image_url, customers.email
  FROM invoices
  JOIN customers ON invoices.customer_id = customers.id
  ORDER BY invoices.date DESC
  LIMIT 5`;
```

- `/app/dashboard/page.tsx`

```
...
import { lusitana } from '@/app/ui/fonts';
import { fetchRevenue, fetchLatestInvoices } from '@/app/lib/data';

export default async function Page() {
  const revenue = await fetchRevenue();
  const latestInvoices = await fetchLatestInvoices();
  return (
   ...
      <div className="mt-6 grid grid-cols-1 gap-6 md:grid-cols-4 lg:grid-cols-8">
        { <RevenueChart revenue={revenue}  /> }
        { <LatestInvoices latestInvoices={latestInvoices} /> }
   ...
  );
}
```

- `/app/ui/dashboard/latest-invoices.tsx` → uncomment all code
- visit http://localhost:3000/dashboard or vercel site

Fetch data for the <Card> components

- Total amount of invoices collected.
- Total amount of invoices pending.
- Total number of invoices.
- Total number of customers.

- `/app/lib/data.ts`

```
...
const invoiceCountPromise = sql`SELECT COUNT(*) FROM invoices`;
const customerCountPromise = sql`SELECT COUNT(*) FROM customers`;
const invoiceStatusPromise = sql`SELECT
     SUM(CASE WHEN status = 'paid' THEN amount ELSE 0 END) AS "paid",
     SUM(CASE WHEN status = 'pending' THEN amount ELSE 0 END) AS "pending"
     FROM invoices`;
const data = await Promise.all([ // Parallel data fetching
  invoiceCountPromise,
  customerCountPromise,
  invoiceStatusPromise,
]);
...
```

→ Example Project → Topics → Data Fetching

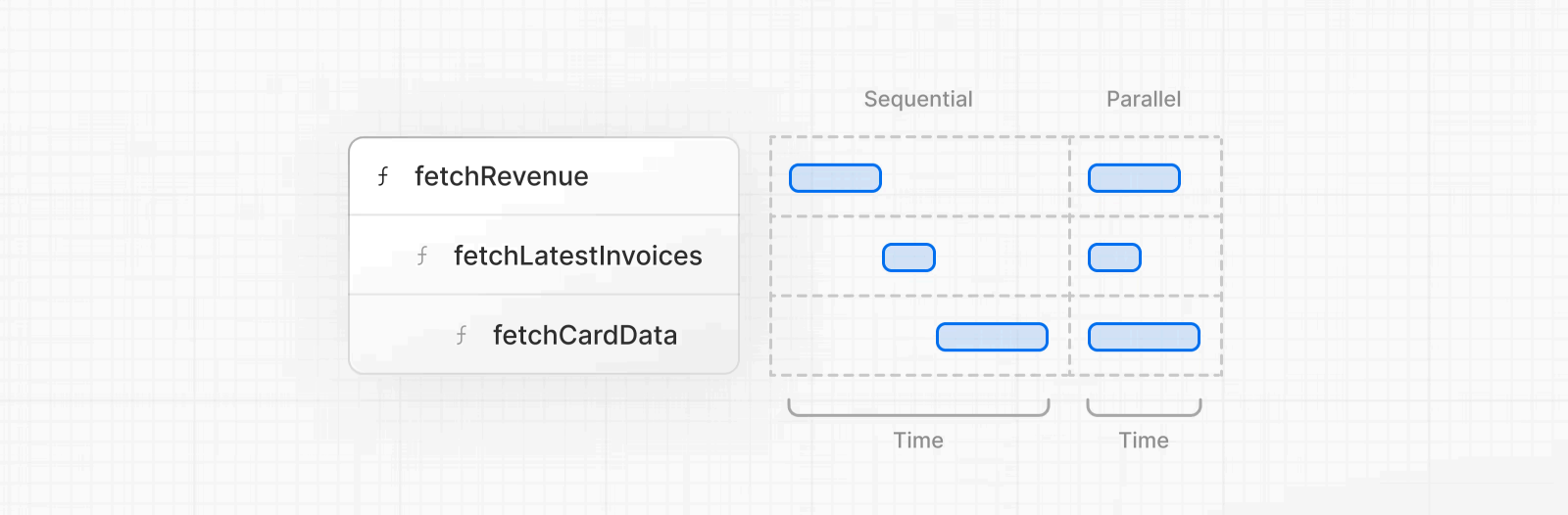
- `/app/dashboard/page.tsx`

```
...
import { fetchRevenue, fetchLatestInvoices, fetchCardData } from '@/app/lib/data';
export default async function Page() {
  const revenue = await fetchRevenue();
  const latestInvoices = await fetchLatestInvoices();
  const { numberOfInvoices, numberOfCustomers, totalPaidInvoices, totalPendingInvoices,} = await fetchCardData();
  return (
   ...
        { <Card title="Collected" value={totalPaidInvoices} type="collected" /> }
        { <Card title="Pending" value={totalPendingInvoices} type="pending" /> }
        { <Card title="Total Invoices" value={numberOfInvoices} type="invoices" /> }
        { <Card title="Total Customers" value={numberOfCustomers} type="customers" />}
    ...
        { <RevenueChart revenue={revenue}  /> }
        { <LatestInvoices latestInvoices={latestInvoices} /> }
    ...
  );
}
```

- visit http://localhost:3000/dashboard 🔗 or vercel site

request waterfalls

- **waterfall** → a sequence of network requests that depend on the completion of previous requests



- *Parallel* data fetching
  - **Promise.all()** or Promise.allSettled() functions to initiate all promises at the same
    - Start executing all data fetches at the same time
  - examine `/app/lib/data.ts`

→ Example Project → Static & Dynamic Rendering

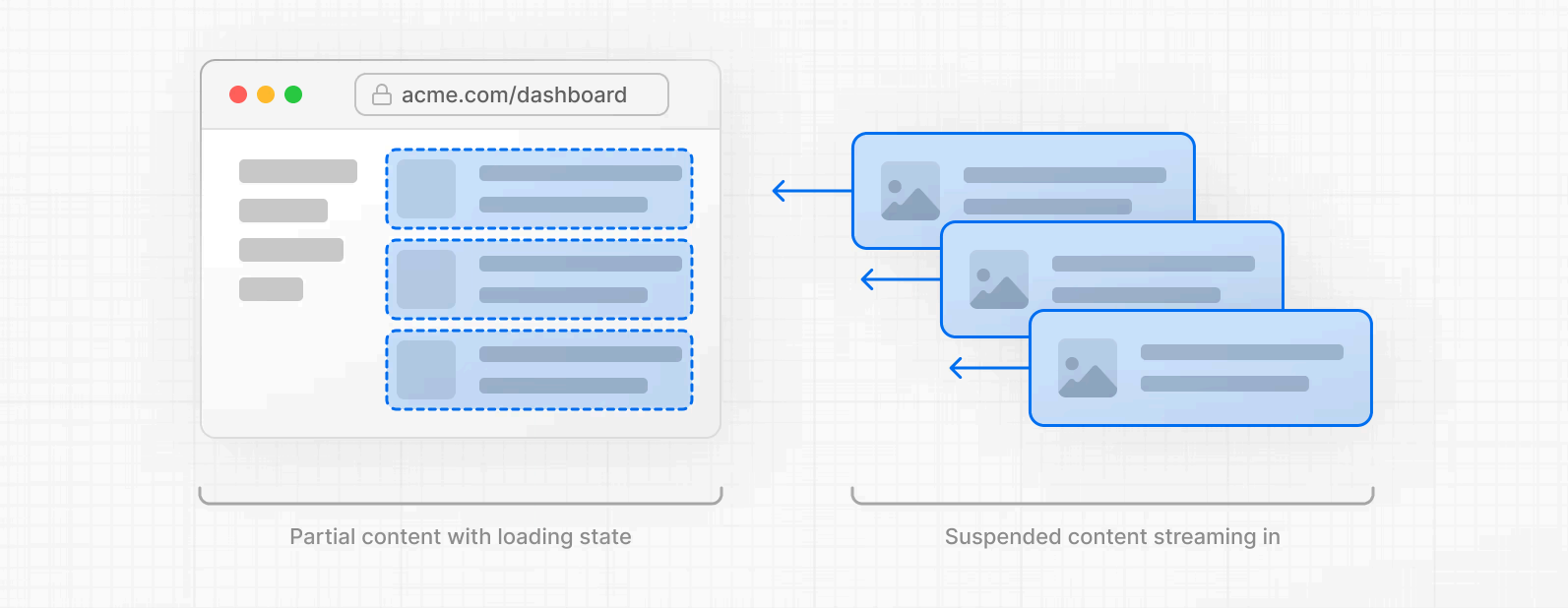
- **Static Rendering** → data fetching and rendering happens on the server at build time (when you deploy)
  - 👍 Faster Websites → Prerendered content can be cached and globally distributed.
  - 👍 Reduced Server Load → the content is cached, your server does not have to dynamically generate content for each user request.
  - 👍 SEO → Prerendered content is easier for search engine crawlers to index
  - 👍 no data or data that is shared across users
  - ❔ personalized data that is regularly updated

→ Example Project → Static & Dynamic Rendering

- **Dynamic Rendering** → content is rendered on the server for each user at request time (when the user visits the page)
  - 👍 Real-Time Data → display real-time or frequently updated data
  - 👍 User-Specific Content → It's easier to serve personalized content
  - 👍 Request Time Information → access information that can only be known at request time, such as cookies or the URL search parameters.

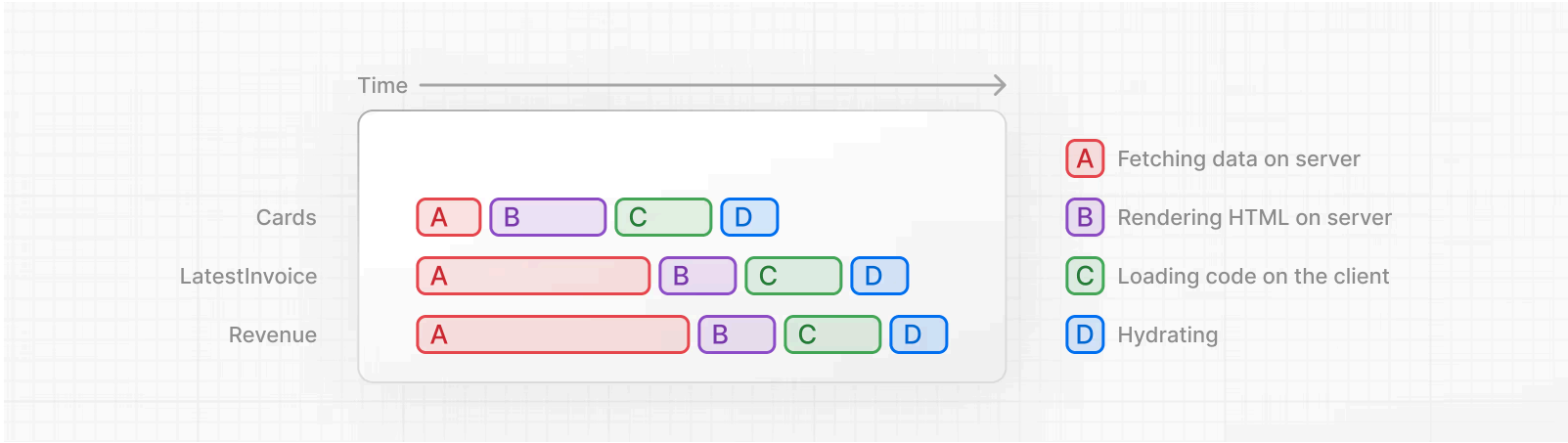
→ Example Project → Topics → Streaming

- Streaming → data transfer technique that allows you to
  - break down a route into smaller "chunks"
  - progressively stream them from the server to the client as they become ready.



→ Example Project → Topics → Streaming

- Streaming → prevent slow data requests from blocking your whole page
  - the user to see and interact with parts of the page without waiting for all the data to load before any UI can be shown to the user



Streaming a whole page with loading

- `/app/dashboard/loading.tsx`

```
export default function Loading() {
  return <div>Loading...</div>;
}
```

- `loading.tsx` → it allows you to create fallback UI to show as a replacement while page content loads.
  - Since `<SideNav>` is static, it's shown immediately.
  - The user can interact with `<SideNav>` while the dynamic content is loading.

> `loading.tsx` → special Next.js file built on top of React Suspense

Streaming a whole page with loading

Adding loading skeletons

- loading skeleton → simplified version of the UI
    - a placeholder (or fallback) to indicate to users that the content is loading
- `/app/dashboard/loading.tsx`

```
import DashboardSkeleton from '@/app/ui/skeletons';

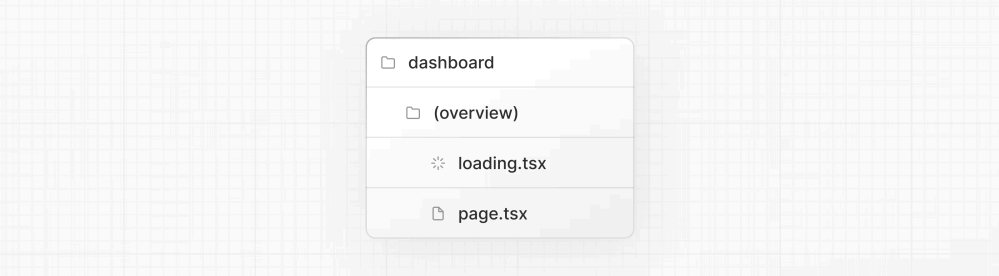
export default function Loading() {
  return <DashboardSkeleton />;
}
```

Streaming a whole page with loading

Adding loading skeletons

- loading skeleton will apply to the invoices and customers pages as well 👎

- Route groups allow you to organize files into logical groups



- ℹ️ A route group can be created by wrapping a folder's name in parenthesis: (folderName)
- `/dashboard/(overview)/page.tsx` becomes `/dashboard`

Streaming a component

- React Suspense → defer rendering parts until some condition is met (e.g. data is loaded)
- `/app/dashboard/(overview)/page.tsx`

```
...
import { lusitana } from '@/app/ui/fonts';
import { fetchLatestInvoices, fetchCardData } from '@/app/lib/data';
import { Suspense } from 'react';
import { RevenueChartSkeleton } from '@/app/ui/skeletons';
export default async function Page() {
  const latestInvoices = await fetchLatestInvoices();
  const { numberOfInvoices, numberOfCustomers, totalPaidInvoices, totalPendingInvoices,} = await fetchCardData();
  return (
...
        <Suspense fallback={<RevenueChartSkeleton />}>
          <RevenueChart />
        </Suspense>
        <LatestInvoices latestInvoices={latestInvoices} />
...
```

Streaming a component

- /app/ui/dashboard/revenue-chart.tsx

```
import { generateYAxis } from '@/app/lib/utils';
import { CalendarIcon } from '@heroicons/react/24/outline';
import { lusitana } from '@/app/ui/fonts';
import { fetchRevenue } from '@/app/lib/data';
// ...
export default async function RevenueChart() { // Make component async, remove the props
  const revenue = await fetchRevenue(); // Fetch data inside the component

  const chartHeight = 350;
  const { yAxisLabels, topLabel } = generateYAxis(revenue);

  if (!revenue || revenue.length === 0) {
    return <p className="mt-4 text-gray-400">No data available.</p>;
  }
 ...
}
```

→ Example Project → Topics → Search and Pagination

- /dashboard/invoices/page.tsx 📁

```
import Pagination from '@/app/ui/invoices/pagination';
import Search from '@/app/ui/search';
import Table from '@/app/ui/invoices/table';
import { CreateInvoice } from '@/app/ui/invoices/buttons';
import { lusitana } from '@/app/ui/fonts';
import { InvoicesTableSkeleton } from '@/app/ui/skeletons';
import { Suspense } from 'react';

export default async function Page() {
  return (
    <div className="w-full">
      <div className="flex w-full items-center justify-between">
        <h1 className={`${lusitana.className} text-2xl`}>Invoices</h1>
      </div>
      <div className="mt-4 flex items-center justify-between gap-2 md:mt-8">
        <Search placeholder="Search invoices..." />
        <CreateInvoice />
      </div>
      {/*  <Suspense key={query + currentPage} fallback={<InvoicesTableSkeleton />}>
        <Table query={query} currentPage={currentPage} />
      </Suspense> */}
      <div className="mt-5 flex w-full justify-center">{/* <Pagination totalPages={totalPages} /> */}</div></div> );
}
```

- `<Search\>` → allows users to search for specific invoices.
- `<Pagination\>` → allows users to navigate between pages of invoices.
- `<Table\>` → displays the invoices.

> When a user searches for an invoice on the client

- the URL params will be updated
- data will be fetched on the server
- the table will re-render on the server with the new data.

URL search params

- Bookmarkable and Shareable URLs → users can bookmark the current state of the application, including their search queries and filters. 👍
- Server-Side Rendering and Initial Load → URL parameters can be directly consumed on the server to render the initial state, making it easier to handle server rendering. 👍
- Analytics and Tracking → easier to track user behavior without requiring additional client-side logic. 👍

Implementation steps

1. Capture the user's input.
2. Update the URL with the search params.
3. Keep the URL in sync with the input field.
4. Update the table to reflect the search query.

Next.js client hooks

- useSearchParams → access the parameters of the current URL.
  - search params for "/dashboard/invoices?page=1&query=pending"
  - {page: '1', query: 'pending'}.
- usePathname → read the current URL's pathname.
- useRouter → Enables navigation between routes within client components programmatically

→ Example Project → Topics → Search and Pagination

1. Capture the user's input

- `/app/ui/search.tsx`

```
'use client';
 import { MagnifyingGlassIcon } from '@heroicons/react/24/outline';

export default function Search({ placeholder }: { placeholder: string }) {
  function handleSearch(term: string) {
    console.log(term);
  }

  return (
    <div className="relative flex flex-1 flex-shrink-0">
      <label htmlFor="search" className="sr-only">
        Search
      </label>
      <input
        className="peer block w-full rounded-md border border-gray-200 py-[9px] pl-10 text-sm outline-2 placeholder:text-gray-500"
        placeholder={placeholder}
        onChange={(e) => {
          handleSearch(e.target.value);
        }}
      />
      <MagnifyingGlassIcon className="absolute left-3 top-1/2 h-[18px] w-[18px] -translate-y-1/2 text-gray-500 peer-focus:text-gray-900" />
    </div>
  );
}
```

2. Update the URL with the search params

- `/app/ui/search.tsx`

```
'use client';
import { MagnifyingGlassIcon } from '@heroicons/react/24/outline';
import { useSearchParams, usePathname, useRouter } from 'next/navigation';
export default function Search({ placeholder }: { placeholder: string }) {
  const searchParams = useSearchParams();
  const pathname = usePathname();
  const { replace } = useRouter();
  function handleSearch(term: string) {
    const params = new URLSearchParams(searchParams);
    //params.set('page', '1');
    if (term) {  params.set('query', term);} else {  params.delete('query'); }
    replace(`${pathname}?${params.toString()}`);
  }
}
```

- `${pathname}` → is the current path ("/dashboard/invoices")
- As the user types into the search bar, params.toString() translates this input into a URL-friendly format.
- **replace(**$pathname?$**{params.toString()})** → updates the URL with the user's search data.
  - `/dashboard/invoices?query=lee` if the user searches for "lee".

3. Keeping the URL and input in sync

- To ensure the input field is in sync with the URL and will be populated when sharing
  - pass a defaultValue to input by reading from searchParams
- `/app/ui/search.tsx`

```
...
<input
  className="peer block w-full rounded-md border border-gray-200 py-[9px] pl-10 text-sm outline-2 placeholder:text-gray-500"
  placeholder={placeholder}
  onChange={(e) => {
    handleSearch(e.target.value);
  }}
  defaultValue={searchParams.get('query')?.toString()}
/>
...
```

4. Updating the table

- `/app/dashboard/invoices/page.tsx`

```
import Pagination from '@/app/ui/invoices/pagination';
import Search from '@/app/ui/search';
import Table from '@/app/ui/invoices/table';
import { CreateInvoice } from '@/app/ui/invoices/buttons';
import { lusitana } from '@/app/ui/fonts';
import { Suspense } from 'react';
import { InvoicesTableSkeleton } from '@/app/ui/skeletons';
export default async function Page({
  searchParams,
}: {
  searchParams?: {
    query?: string;   page?: string;
  };
}) {
  const query = searchParams?.query || '';
  const currentPage = Number(searchParams?.page) || 1;
   return (
    <div className="w-full"><div className="flex w-full items-center justify-between"><h1 className={`${lusitana.className} text-2xl`}>Invoices</h1> </div>
      <div className="mt-4 flex items-center justify-between gap-2 md:mt-8">
        <Search placeholder="Search invoices..." /><CreateInvoice /> </div>
      <Suspense key={query + currentPage} fallback={<InvoicesTableSkeleton />}>
        <Table query={query} currentPage={currentPage} />
      </Suspense>
      <div className="mt-5 flex w-full justify-center">{/* <Pagination totalPages={totalPages} /> */} </div></div> );
}
```

Adding Pagination

- `/app/dashboard/invoices/page.tsx`

```
// ...
import { fetchInvoicesPages } from '@/app/lib/data';
export default async function Page({
  searchParams,
}: {
  searchParams?: { query?: string,  page?: string, },
}) {
  const query = searchParams?.query || '';
  const currentPage = Number(searchParams?.page) || 1;
  const totalPages = await fetchInvoicesPages(query);
  return (
    // ...
        { <Pagination totalPages={totalPages} /> }
  );
}
```

Adding Pagination

- `/app/ui/invoices/pagination.tsx`

```tsx
'use client';

import { ArrowLeftIcon, ArrowRightIcon } from '@heroicons/react/24/outline';
import clsx from 'clsx';
import Link from 'next/link';
import { generatePagination } from '@/app/lib/utils';
import { usePathname, useSearchParams } from 'next/navigation';

export default function Pagination({ totalPages }: { totalPages: number }) {
  const pathname = usePathname();
  const searchParams = useSearchParams();
  const currentPage = Number(searchParams.get('page')) || 1;

  const createPageURL = (pageNumber: number | string) => {
    const params = new URLSearchParams(searchParams);
    params.set('page', pageNumber.toString());
    return `${pathname}?${params.toString()}`;
  };

  // ...
}
```

- `createPageURL` →
  - creates an instance of the current search parameters
  - it updates the "page" parameter to the provided page number
  - it constructs the full URL using the pathname and updated search parameters.

Using forms with Server Actions

- React → action attribute in the <form> element to invoke actions
  - The action will automatically receive the native FormData object, containing the captured data.

```
// Server Component
export default function Page() {
  // Action
  async function create(formData: FormData) {
    'use server';

    // Logic to mutate data...
  }

  // Invoke the action using the "action" attribute
  return <form action={create}>...</form>;
}
```

Create invoice

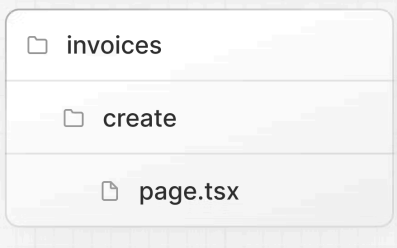
Steps

1. Create a form to capture the user's input.
2. Create a Server Action and invoke it from the form.
3. Inside your Server Action, extract the data from the formData object.
4. Validate and prepare the data to be inserted into your database.
5. Insert the data and handle any errors.
6. Revalidate the cache and redirect the user back to invoices page.

Create invoice

1. Create a new route and form

- add a new route & page



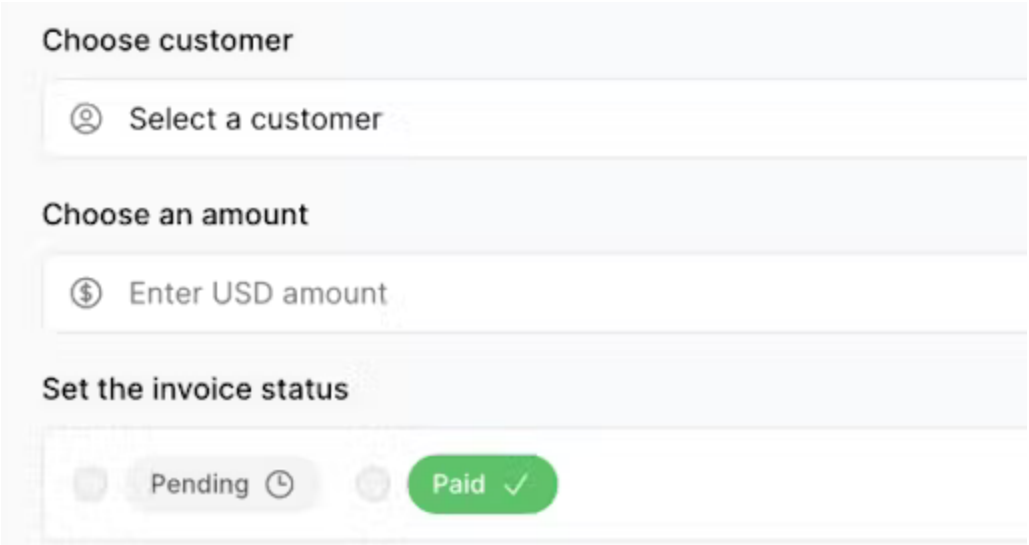
- `/dashboard/invoices/create/page.tsx`

```
import Form from '@/app/ui/invoices/create-form';
import Breadcrumbs from '@/app/ui/invoices/breadcrumbs';
import { fetchCustomers } from '@/app/lib/data';
export default async function Page() {
  const customers = await fetchCustomers();
  return (    <main>
      <Breadcrumbs
        breadcrumbs={[
          { label: 'Invoices', href: '/dashboard/invoices' },
          { label: 'Create Invoice', href: '/dashboard/invoices/create',
            active: true, },
        ]}
      />
      <Form customers={customers} />    </main>  );
}
```

Create invoice

1. Create a new route and form

- `<Form>` component
  - Has one `<select>` (dropdown) element with a list of customers.
  - Has one `<input>` element for the amount with type="number".
  - Has two `<input>` elements for the status with type="radio".
  - Has one `<button>` with type="submit".



Create invoice

2. Create a Server Action

- `/app/lib/actions.ts`

```
'use server';
export async function createInvoice(formData: FormData) {}
```

- `/app/ui/invoices/create-form.tsx`

```
...
import { createInvoice } from '@/app/lib/actions';
...
export default function Form({ customers }: { customers: CustomerField[] }) {
  return (
    <form action={createInvoice}>
...
```

> Server Actions create a POST API endpoint

Create invoice

3. Extract the data from formData

- `/app/lib/actions.ts` 📁

```
'use server';
export async function createInvoice(formData: FormData) {
  const rawFormData = {
    customerId: formData.get('customerId'),
    amount: formData.get('amount'),
    status: formData.get('status'),
  };
  // Test it out - Check Server log (not Client !)
  console.log(rawFormData);
}
```

> ℹ️ check FormData docs 📖🔗

Create invoice

4. Validate and prepare the data

- `/app/lib/definitions.ts`

```
export type Invoice = {
  id: string; // Will be created on the database
  customer_id: string;
  amount: number; // Stored in cents
  status: 'pending' | 'paid';
  date: string;
};
```

Create invoice

4. Validate and prepare the data

- Type validation
- `/app/lib/actions.ts`

```
'use server';
import { z } from 'zod';
const FormSchema = z.object({
  id: z.string(),
  customerId: z.string(),
  amount: z.coerce.number(),
  status: z.enum(['pending', 'paid']),
  date: z.string(),
});
const CreateInvoice = FormSchema.omit({ id: true, date: true });
export async function createInvoice(formData: FormData) {
  // ...
};
```

```
export async function createInvoice(formData: FormData) {
  const { customerId, amount, status } = CreateInvoice.parse({
    customerId: formData.get('customerId'),
    amount: formData.get('amount'),
    status: formData.get('status'),
  });
  const amountInCents = amount * 100;
  const date = new Date().toISOString().split('T')[0];
}
```

> Zod a TypeScript validation library

Create invoice

5. Inserting the data into your database

- `/app/lib/actions.ts`

```
...
import { sql } from '@vercel/postgres';
...
export async function createInvoice(formData: FormData) {
  const { customerId, amount, status } = CreateInvoice.parse({
    customerId: formData.get('customerId'),  amount: formData.get('amount'),
    status: formData.get('status'),  });
  const amountInCents = amount * 100;
  const date = new Date().toISOString().split('T')[0];
  await sql` INSERT INTO invoices (customer_id, amount, status, date)
             VALUES (${customerId}, ${amountInCents}, ${status}, ${date})`; }
```

Create invoice

6. Revalidate and redirect

- `/app/lib/actions.ts`

```
...
import { revalidatePath } from 'next/cache';
import { redirect } from 'next/navigation';
...
export async function createInvoice(formData: FormData) {
...
  await sql`
    INSERT INTO invoices (customer_id, amount, status, date)
    VALUES (${customerId}, ${amountInCents}, ${status}, ${date})
  `;
  revalidatePath('/dashboard/invoices');
  redirect('/dashboard/invoices');
}
```

Update invoice

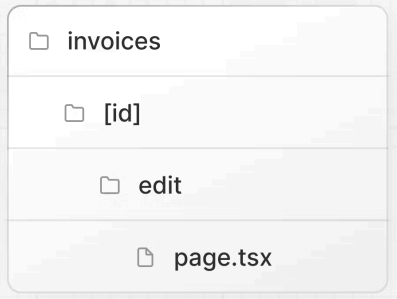
Steps

1. Create a new dynamic route segment with the invoice id.
2. Read the invoice id from the page params.
3. Fetch the specific invoice from your database.
4. Pre-populate the form with the invoice data.
5. Update the invoice data in your database.

Update invoice

1. Create a new dynamic route segment with the invoice id.

- add a new route & page



- `/app/ui/invoices/table.tsx`

```tsx
export default async function InvoicesTable({
  query,
  currentPage,
}: {
  query: string;
  currentPage: number;
}) {
  return (
    // ...
    <td className="flex justify-end gap-2 whitespace-nowrap px-6 py-4 text-sm">
      <UpdateInvoice id={invoice.id} />
      <DeleteInvoice id={invoice.id} />
    </td>
    // ...
  );
}
```

Update invoice

1. Create a new dynamic route segment with the invoice id.

- `/app/ui/invoices/buttons.tsx`

```
mport { PencilIcon, PlusIcon, TrashIcon } from '@heroicons/react/24/outline';
import Link from 'next/link';

// ...
export function UpdateInvoice({ id }: { id: string }) {
  return (
    <Link
      href={`/dashboard/invoices/${id}/edit`}
      className="rounded-md border p-2 hover:bg-gray-100"
    >
      <PencilIcon className="w-5" />
    </Link>  );}
```

Update invoice

2. Read the invoice id from page params

- `/app/dashboard/invoices/[id]/edit/page.tsx`

```
import Form from '@/app/ui/invoices/edit-form';
import Breadcrumbs from '@/app/ui/invoices/breadcrumbs';
import { fetchCustomers } from '@/app/lib/data';
export default async function Page({ params }: { params: { id: string } }) {
  const id = params.id;
  return (
    <main>      <Breadcrumbs         breadcrumbs={[
          { label: 'Invoices', href: '/dashboard/invoices' },
          {
            label: 'Edit Invoice',
            href: `/dashboard/invoices/${id}/edit`,
            active: true,
          },        ]}      />
      <Form invoice={invoice} customers={customers} />    </main>  );
}
```

Update invoice

3. Fetch the specific invoice

- `/app/dashboard/invoices/[id]/edit/page.tsx`

```
import Form from '@/app/ui/invoices/edit-form';
import Breadcrumbs from '@/app/ui/invoices/breadcrumbs';
import { fetchInvoiceById, fetchCustomers } from '@/app/lib/data';

export default async function Page({ params }: { params: { id: string } }) {
  const id = params.id;
  const [invoice, customers] = await Promise.all([
    fetchInvoiceById(id),
    fetchCustomers(),
  ]);
  // ...
}
```

Update invoice

4. Pass the id to the Server Action

- `/app/ui/invoices/edit-form.tsx` 📁 → 🛑

```
// Passing an id as argument won't work
<form action={updateInvoice(id)}>
```

- `/app/ui/invoices/edit-form.tsx` 📁

```
// ...
import { updateInvoice } from '@/app/lib/actions';
export default function EditInvoiceForm({
  invoice,  customers,
}: {
  invoice: InvoiceForm;   customers: CustomerField[];
}) {
  const updateInvoiceWithId = updateInvoice.bind(null, invoice.id);
  return (<form action={updateInvoiceWithId}><input type="hidden" name="id" value={invoice.id} /></form>);
}
```

Update invoice

4. Pass the id to the Server Action

- `/app/lib/actions.ts`

```
// Use Zod to update the expected types
const UpdateInvoice = FormSchema.omit({ id: true, date: true });
// ...
export async function updateInvoice(id: string, formData: FormData) {
  const { customerId, amount, status } = UpdateInvoice.parse({
    customerId: formData.get('customerId'),
    amount: formData.get('amount'), status: formData.get('status'),
  });
  const amountInCents = amount * 100;
  await sql`
    UPDATE invoices SET customer_id = ${customerId}, amount = ${amountInCents}, status = ${status}
    WHERE id = ${id}
  `;
  revalidatePath('/dashboard/invoices'); //clear the client cache and make a new server request.
  redirect('/dashboard/invoices'); //redirect the user to the invoice's page.
}
```

Delete invoice

- wrap the delete button in a `<form>` element and pass the id to the Server Action using bind
- `/app/ui/invoices/buttons.tsx`

```tsx
import { deleteInvoice } from '@/app/lib/actions';
// ...
export function DeleteInvoice({ id }: { id: string }) {
  const deleteInvoiceWithId = deleteInvoice.bind(null, id);
  return (
    <form action={deleteInvoiceWithId}>
      <button className="rounded-md border p-2 hover:bg-gray-100">
        <span className="sr-only">Delete</span>
        <TrashIcon className="w-4" />
      </button>
    </form>
  );
}
```

- `/app/lib/actions.ts`

```ts
export async function deleteInvoice(id: string) {
  await sql`DELETE FROM invoices WHERE id = ${id}`;
  revalidatePath('/dashboard/invoices');
}
```

Next Steps

- `Handling Errors` → How to handle general and 404 not found errors.
- `Accessibility` → How to do server-side form validation and tips for improving accessibility.
- `Authentication` → How to add authentication to your application using NextAuth.js
- `Metadata` → How to add metadata and prepare your application for social sharing.

Final Code

- You can find the final code for this example https://github.com/vercel/next-learn/tree/main/dashboard

→ Thank you!

Ερωτήσεις ?